

Intermediate (chunking method changes the answer) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below use the same test document:

```
DOCUMENT = """Coffee brewing starts with the grind size. A coarse grind works best for
Water temperature matters just as much as grind size. Water that is too hot, above
The final step is the pour itself. A slow, steady pour in a spiral pattern wets a
QUESTION = "What water temperature should be avoided when brewing coffee?"
```

Basic Version

Approach 1 — the direct way

```
# chunking_practice.py – Intermediate section
import math
from openai import OpenAI

client = OpenAI()

def chunk_by_chars(text, chunk_size=200):
    chunks = []
    for i in range(0, len(text), chunk_size):
        chunks.append(text[i:i+chunk_size])
    return chunks

def chunk_by_paragraph(text):
    chunks = []
    for piece in text.split("\n\n"):
        stripped = piece.strip()
        if stripped:
            chunks.append(stripped)
    return chunks
```

```

def get_embedding(text):
    response = client.embeddings.create(model="text-embedding-3-small", input=text)
    return response.data[0].embedding

def cosine_similarity(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = math.sqrt(sum(x * x for x in a))
    norm_b = math.sqrt(sum(x * x for x in b))
    return dot / (norm_a * norm_b)

def find_best_chunk(chunks, question_vector):
    best_chunk = None
    best_score = -1
    for chunk in chunks:
        chunk_vector = get_embedding(chunk)
        score = cosine_similarity(question_vector, chunk_vector)
        if score > best_score:
            best_score = score
            best_chunk = chunk
    return best_chunk, best_score

question_vector = get_embedding(QUESTION)

char_chunks = chunk_by_chars(DOCUMENT)
paragraph_chunks = chunk_by_paragraph(DOCUMENT)

char_best, char_score = find_best_chunk(char_chunks, question_vector)
paragraph_best, paragraph_score = find_best_chunk(paragraph_chunks, question_vector)

print("chunk_by_chars best match, score:", char_score)
print(char_best)
print()
print("chunk_by_paragraph best match, score:", paragraph_score)
print(paragraph_best)

```

Expected output: with `chunk_size=200`, `chunk_by_chars` cuts this document right through the word "scalds" — one chunk ends "...205 degrees Fahrenheit, sc" and the next one starts "alds the coffee...". Whichever half wins, it's missing part of the answer.

`chunk_by_paragraph` keeps the whole "Water temperature matters..." paragraph — including "205 degrees Fahrenheit" and the reason why — as one unbroken chunk.

This version works correctly for what the exercise asks. It's missing type hints and reuses the same `get_embedding` call for every chunk on every run, with no reuse across calls.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints and a `find_best_chunk` helper

```
# chunking_practice.py – Intermediate section
import math
from openai import OpenAI

client = OpenAI()

DOCUMENT: str = "..." # as above
QUESTION: str = "What water temperature should be avoided when brewing coffee?"

def chunk_by_chars(text: str, chunk_size: int = 200) -> list[str]:
    chunks = []
    for i in range(0, len(text), chunk_size):
        chunks.append(text[i:i + chunk_size])
    return chunks

def chunk_by_paragraph(text: str) -> list[str]:
    chunks = []
    for piece in text.split("\n\n"):
        stripped = piece.strip()
        if stripped:
            chunks.append(stripped)
    return chunks

def get_embedding(text: str) -> list[float]:
    response = client.embeddings.create(model="text-embedding-3-small", input=text)
    return response.data[0].embedding

def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = math.sqrt(sum(x * x for x in a))
    norm_b = math.sqrt(sum(x * x for x in b))
    return dot / (norm_a * norm_b)
```

```

def find_best_chunk(chunks: list[str], question_vector: list[float]) -> tuple[str, float]:
    best_chunk = ""
    best_score = -1.0
    for chunk in chunks:
        chunk_vector = get_embedding(chunk)
        score = cosine_similarity(question_vector, chunk_vector)
        if score > best_score:
            best_score = score
            best_chunk = chunk
    return best_chunk, best_score

def main() -> None:
    question_vector = get_embedding(QUESTION)
    methods = {
        "chunk_by_chars": chunk_by_chars(DOCUMENT),
        "chunk_by_paragraph": chunk_by_paragraph(DOCUMENT),
    }
    for name, chunks in methods.items():
        best_chunk, best_score = find_best_chunk(chunks, question_vector)
        print(f"{name} best match, score: {best_score:.4f}")
        print(best_chunk)
        print()

if __name__ == "__main__":
    main()

```

Expected output: same conclusion as Basic — `chunk_by_paragraph` 's top match contains the whole temperature fact; `chunk_by_chars` 's top match is missing half of "scalds."

Difference from Basic: full type hints on every function. `find_best_chunk` has an explicit `tuple[str, float]` return type, so any caller knows exactly what comes back without reading its body. The two methods are driven from one `methods` dict and one loop in `main()`, instead of copy-pasted print statements for each — adding a third chunking method later means adding one dict entry, not duplicating the comparison logic again.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — overlapping fixed-size chunks

```
# chunking_practice.py – Intermediate section
def chunk_by_chars_with_overlap(text: str, chunk_size: int = 200, overlap: int =
    chunks: list[str] = []
    step = chunk_size - overlap
    i = 0
    while i < len(text):
        chunks.append(text[i:i + chunk_size])
        i += step
    return chunks
```

Run this against `DOCUMENT` with `chunk_size=200`, `overlap=40` and look at the chunk boundaries: the word "scalds" that got split in half by the non-overlapping version now appears whole in at least one chunk, because each new chunk starts 40 characters *before* the previous one ended. The word that was previously exactly on the cut line is now safely inside the overlapping region instead of split by it.

Approach 2 — a paragraph chunker that doesn't silently give up

```
# chunking_practice.py – Intermediate section
def chunk_by_paragraph_safe(text: str) -> list[str]:
    pieces = chunk_by_paragraph(text)
    if len(pieces) <= 1:
        # no real paragraph breaks in this document -- fall back instead of
        # silently returning the whole document as "1 chunk"
        return chunk_by_chars_with_overlap(text, chunk_size=200, overlap=40)
    return pieces

# proof this matters: a document with no blank lines at all
single_block = DOCUMENT.replace("\n\n", " ")

naive_result = chunk_by_paragraph(single_block)
safe_result = chunk_by_paragraph_safe(single_block)
```

```
print("naive chunk_by_paragraph on a no-blank-line document:", len(naive_result),
print("chunk_by_paragraph_safe on the same document:", len(safe_result), "chunk(s)
```

Expected output:

```
naive chunk_by_paragraph on a no-blank-line document: 1 chunk(s)
chunk_by_paragraph_safe on the same document: 4 chunk(s)
```

The plain `chunk_by_paragraph` hands back the entire document as a single "chunk" — no error, no warning, just a search index that's secretly not chunked at all. `chunk_by_paragraph_safe` notices that and falls back to the overlapping character chunker instead.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's two chunkers both produce clean, non-overlapping pieces, and `chunk_by_paragraph` trusts that the document has real paragraph breaks to split on. Approach 1 fixes the boundary problem directly: overlap means a fact sitting near a cut usually survives whole in at least one chunk, without needing a bigger `k` at search time to compensate (the workaround `chunk_boundary_split`'s solution describes). Approach 2 fixes a different, quieter failure — a document that defeats `chunk_by_paragraph` entirely by having no blank lines — by detecting that case and falling back to Approach 1's chunker instead of returning one useless giant chunk.

Which one should you actually write? For a document set you've actually looked at (like this exercise's `DOCUMENT`), Intermediate's plain `chunk_by_paragraph` is fine — real paragraph breaks, no boundary problem to speak of. The moment your documents come from somewhere you don't fully control — scraped pages, converted PDFs, user uploads — reach for Advanced Approach 2's fallback, since you can't guarantee every file will have the blank-line structure you're assuming. Reach for Approach 1's overlap specifically for `chunk_by_chars`-style fixed-size chunking, any time a fact could plausibly land near a chunk boundary — which, over a large enough document set, it eventually will.